

AIXMOS Agent Network — User Guide

A 10-agent operations toolkit for TMMT / AIXMOS. Runs as Node.js processes on Windows, talks to either Claude (cloud) or Ollama (local) for reasoning. This guide covers daily use and how to extend the system.

1. Quick Start

You have a USB labelled **CYBORG** (drive letter varies — usually **D:** or **E:**). The agents live in `<USB>\AIXMOS-AGENTS\`.

The 3-second path

1. Plug the USB into a Windows PC.
2. Open the `AIXMOS-AGENTS` folder.
3. Double-click **START-HERE.bat**.
4. Pick one of:

- **1 — INSTALL** → copies the agents to `C:\Users\<you>\AIXMOS`, creates Desktop launchers. *Best for the laptop you actually work on.* - **2 — RUN FROM USB** → runs agents directly off the USB. *Best for borrowed/temporary PCs.* - **3 — OFFLINE MODE** → installs Ollama + local model so the agents work without internet. *Optional.*

After install, your Desktop has:

- `AIXMOS.bat` — master orchestrator menu (all 10 agents).
- `AIXMOS-LAUNCHERS\` — one `.bat` per agent for one-click runs.

Smoke test

Double-click `Desktop\AIXMOS-LAUNCHERS\RUN-AGENT-TESTS.bat`. Expect **47/49 pass** out of the box. The 2 failures are environment-only:

- `tank:docker-available` — Docker Desktop not running (TANK `--up` needs it).
- `sticks:env` — `SUPABASE_URL` not set (STICKS `--scan` needs Supabase).

Both are fine to leave for now — agents respond fine without them.

2. The 10 Agents

Every agent has a system prompt in `agents/prompts.js`. They share a `BUSINESS_CONTEXT` block (revenue is car rentals via TMMT Auto Services; Level A escalation; owner-only escalations for insurance, unpleasant CX, and pitfalls).

CHUMMO — Customer comms

*Communicates **H**uman-first **U**nified **M**obility **M**ember **O**perations.*

Drafts every outbound customer message in your voice. Friend-first, no corporate openers, name first, one CTA, SMS under 160 chars when possible. Output is *only* the message; CHUMMO ends with a `MOOSE_HANDOFF:` line telling MOOSE what internal task this triggers.

- **Launcher:** `Desktop\AIXMOS-LAUNCHERS\CHUMMO.bat`
- **Direct:** `node orchestrator.js chummo`

MOOSE — Ops execution

*Moves **O**perations **O**rchestrates **S**yncs **E**xecutes.*

Executes reversible internal ops without hesitation. Returns numbered tasks with an owner + timing (`TODAY` / `24H` / `THIS WEEK`) on every line. Ends with `CAPTAIN_HANDOFF:` .

- **Launcher:** `MOOSE.bat`
- **Direct:** `node moose.js`

CAPTAIN — Command & routing

*Command **A**uthority for **P**riorities, **T**iming, **A**nd **N**avigation.*

Moves the rubik's-cube: cars, money, agents, customers, deals, docs, time. Outputs a `COMMAND BRIEF` with priority stack, resource moves, team routing, and a 🚨 escalate-to-owner block (only if insurance / bad CX / pitfalls).

- **Launcher:** `CAPTAIN.bat`
- **Direct:** `node captain.js`

WONDER WOMAN — Trust defender

*Watch **O**ver **N**eeds, **D**efend, **E**scalate, **R**esolve.*

Fixes trust-breaking situations. Moral check + long-term-fit check before any potentially harmful action. Says "This is not a good idea" when it isn't. Output line: `ACTION | MESSAGE | SEND VIA | TIMING | OWNER | IF NO RESPONSE` .

- **Launcher:** `WONDER-WOMAN.bat`
- **Direct:** `node wonderwoman.js`

VISION — Governance / go-no-go

Long-term-fit reviewer. Verdict is one of `GO` ✅ / `HOLD` ⚠️ / `NO-GO` ❌ with a one-line reason and the long-term-fit assessment. If `HOLD` , lists what must be true to proceed.

- **Launcher:** `VISION.bat`
- **Direct:** `node vision.js`

JARVIS — Speaks for the owner

The orchestrator that talks for Muhammad Taha so he stops repeating himself. Explains payouts, the business model, day-to-day, bottlenecks — in his voice (direct, respectful, no fluff). Classifies an incoming request, names primary + support agents, gives ordered execution steps. Does **not** vote on the 5-panel.

- **Launcher:** `JARVIS.bat`
- **Direct:** `node jarvis.js`

TANK — Heavy infrastructure

Owns Docker hub-brain, n8n, Supabase / tmmt-os, Vercel deploys, migrations, env wiring. Outputs numbered steps with exact commands. Flags blockers. Ends with `BOB_HANDOFF:` .

- **Launchers:** `TANK.bat` (menu), or directly: `node tank.js --up / --down / --status`
- **Requires:** Docker Desktop running for `--up` .

FLY GUY — Customer comms support

Drafts under CHUMMO voice rules but never overrides CHUMMO. One CTA, never corporate. Ends with `CHUMMO_HANDOFF:` so CHUMMO can polish or queue for Level A.

- **Launcher:** `FLY-GUY.bat`
- **Direct:** `node flyguy.js`

BOB — Audit & docs

Logs what happened, what was decided, who owns follow-up. Outputs `AUDIT LOG` + `DOC UPDATES NEEDED` (bullets). Use this when you want a paper trail of an agent decision.

- **Launcher:** `BOB.bat`
- **Direct:** `node bob.js`

STICKS — SLA & monitoring

Rentals-first SLA dashboard. Tracks overdue and stuck requests across  CRITICAL /  AT RISK /  ON TRACK. Ends with `CAPTAIN_HANDOFF:` when routing is needed.

- **Launchers:** `STICKS.bat` (menu), `node sticks.js --scan` (Supabase scan), `node sticks.js --push-ghl` (GHL webhook).
- **Requires:** `SUPABASE_URL` + `SUPABASE_SERVICE_ROLE_KEY` for `--scan` .

Other launchers

- `MORNING-BRIEF.bat` — daily briefing (uses `briefing.js`). Scheduled at 7 AM by the installer if you opted in.
 - `BRAIN-ASK.bat` / `BRAIN-STATUS.bat` — central "AI brain" helpers (`brain-ask.js` , `brain-status.js`).
 - `RUN-AGENT-TESTS.bat` — full smoke test (`test-all-agents.js --live`).
-

3. Installation

Plug-and-play (recommended)

1. Plug the CYBORG USB into the PC.
2. Open `AIXMOS-AGENTS\` and double-click `START-HERE.bat`.
3. Choose mode 1 (INSTALL).
4. When asked, paste your `ANTHROPIC_API_KEY` (get one at <https://console.anthropic.com>).
5. Optionally schedule the 7 AM morning brief.

The installer:

- Mirrors the entire tree to `%USERPROFILE%\AIXMOS` (via `robocopy /MIR`, preserving `aixmos-state.json`).
- Confirms `node_modules` exists or runs `npm install`.
- Saves the API key with `setx` (User scope; survives reboots).
- Creates `Desktop\AIXMOS.bat` + 14 launchers in `Desktop\AIXMOS-LAUNCHERS\`.
- Optionally creates a Task Scheduler entry for the morning brief.

Manual install

```
robocopy D:\AIXMOS-AGENTS C:\Users\<you>\AIXMOS /MIR /XF aixmos-state.json
cd C:\Users\<you>\AIXMOS
npm install
setx ANTHROPIC_API_KEY "sk-ant- ..."
node test-all-agents.js --live
```

Prerequisites

Required	Notes
Node.js LTS	Use the bundled <code>_installers\node-v24.15.0-x64.msi</code> if missing.
ANTHROPIC_API_KEY	Set via <code>setx</code> or paste during installer. Not needed if you only use offline mode.
Internet	Required for Claude backend. Not required for Ollama backend.
Optional: Docker Desktop	Needed only for TANK <code>--up</code> (spinning up hub-brain).
Optional: Supabase creds	Needed only for STICKS <code>--scan</code> .
Optional: GHL webhook secret	Needed only for STICKS <code>--push-ghl</code> .
Optional: Ollama runtime	Needed only for offline LLM mode.

4. Online vs Offline LLM mode

The agents pick a backend at runtime from `AIXMOS_LLM_BACKEND` :

Value	What it does
<code>claude</code> (default)	Anthropic cloud API. Uses <code>ANTHROPIC_API_KEY</code> .
<code>ollama</code>	Local Ollama runtime at <code>OLLAMA_HOST</code> (default <code>http://localhost:11434</code>). Model name from <code>OLLAMA_MODEL</code> .
<code>auto</code>	Tries Claude first; falls back to Ollama if cloud fails.

Quality vs cost tradeoff

Aspect	Claude Sonnet 4.6	Llama 3.1 8B	Phi-3 Mini
Voice/brand quality (CHUMMO, FLY GUY)	★★★★★	★★★	★★
Analytical quality (BOB, STICKS, VISION)	★★★★★	★★★★★	★★★
Speed per response	2–5s	10–25s (CPU) / 2–8s (GPU)	3–10s (CPU)
Disk size	none	~4.7 GB	~2.3 GB
Internet	required	none after pull	none after pull
First-time setup	API key only	install Ollama + <code>ollama pull llama3.1:8b</code>	install Ollama + <code>ollama pull phi3:mini</code>

Recommendation: stay on `claude` for daily customer-facing work; switch to `ollama` only when offline. Set `AIXMOS_LLM_BACKEND=auto` for hands-free fallback.

Switching modes

```
# Permanent (User env, survives reboot)
setx AIXMOS_LLM_BACKEND "ollama"
setx OLLAMA_MODEL "llama3.1:8b"

# Session-only
$env:AIXMOS_LLM_BACKEND = "claude"
```

Or run `OFFLINE-MODE-SETUP.bat` from the USB to do install + config in one go.

FAT32 model-file limit

The CYBORG USB is FAT32 → 4 GB per file. **Phi-3 Mini fits, Llama 8B does not.** Recommended layout: keep Phi-3 on the USB for portable use; install Llama 3.1 8B to C: via Ollama (it stores models in

%USERPROFILE%\ollama\models , no 4 GB limit).

5. Configuration

Env file locations (loaded by `lib/env.js`)

In order of precedence (later files do **not** override earlier ones):

1. `<install>\.env`
2. `<install>\config\aixmos.env`
3. `<tmtt_os>\.env.local` (path from `agents/registry.json::infrastructure_paths.tmtt_os`)
4. `<tmtt_os>\.env`
5. `<tmtt_os>\..\AUTOMATIONS\.env`

Process env (`setx` / `$env:`) wins over all of these.

Env vars — full list

```
# Claude (online)
ANTHROPIC_API_KEY=sk-ant- ...
AIXMOS_MODEL=claude-sonnet-4-6

# Backend selector
AIXMOS_LLM_BACKEND=claude           # claude | ollama | auto

# Ollama (offline)
OLLAMA_HOST=http://localhost:11434
OLLAMA_MODEL=llama3.1:8b           # or phi3:mini

# TMMT OS / production
TMMT_OPS_URL=https://your-tmtt-os.vercel.app
AGENT_WEBHOOK_SECRET=

# Supabase (STICKS --scan)
SUPABASE_URL=https://<project>.supabase.co
SUPABASE_SERVICE_ROLE_KEY=

# GHL webhook (STICKS --push-ghl)
GHL_OVERDUE_WEBHOOK_SECRET=

# Debug
AIXMOS_DEBUG=1                     # prints LLM fallback messages
```

State file — `aixmos-state.json`

Tracks per-agent state across sessions: last lead, last mode, last output snippet, active handoffs. Persisted at the install root. **The installer preserves this file when refreshing from the USB** (`robocopy /XF aixmos-state.json`).

Registry — `agents/registry.json`

The single source of truth for which agents exist, which script powers each, and which paths point to TMMT-OS infrastructure. The smoke test (`test-all-agents.js`) reads it to verify every registered agent has a matching script.

System prompts — `agents/prompts.js`

CommonJS module that exports one prompt string per agent. Shared `BUSINESS_CONTEXT` block lives at the top.

6. Troubleshooting

Symptom	Likely cause	Fix
<code>404 not_found_error model: claude- ...</code>	Hard-coded outdated model id	Set <code>AIXMOS_MODEL=claude-sonnet-4-6</code> or update the literal.
<code>ANTHROPIC_API_KEY is not set</code>	Env var missing	<code>setx ANTHROPIC_API_KEY "sk-ant- ... "</code> then open a NEW terminal.
<code>Ollama not reachable at http://localhost:11434</code>	Ollama service not running	<code>ollama serve</code> or restart the Ollama tray icon.
<code>Docker not running - TANK --up will fail</code>	Docker Desktop stopped	Start Docker Desktop, wait for the whale icon to settle.
<code>Missing SUPABASE_URL - --scan needs Supabase</code>	Supabase env vars unset	Put them in <code>&lt;install&gt;\.env</code> or <code>tmmt-os\.env.local</code> .
<code>global fetch missing - Ollama backend needs Node.js 18+</code>	Old Node	Upgrade to Node 20+ LTS.
<code>Cannot find module '@anthropic-ai/sdk'</code>	<code>npm install</code> never ran	<code>cd &lt;install&gt; &amp;&amp; npm install</code> .
Smoke test shows 0 live passes	Network blocked or wrong key	Try a <code>curl https://api.anthropic.com</code> to confirm reachability.

Where to look first when something breaks

1. `node test-all-agents.js --live` — full smoke test, names the broken thing.

2. `AIXMOS_DEBUG=1 node <agent>.js` — verbose LLM path.
 3. `aixmos-state.json` — last known good state of every agent.
-

7. Updating from the USB

When you change agent code (or the USB has a newer version):

```
robocopy D:\AIXMOS-AGENTS C:\Users\<you>\AIXMOS /MIR /XF aixmos-state.json
```

That preserves your local state, mirrors everything else. Or just re-run `START-HERE.bat → 1 (INSTALL)`.

The installer is idempotent — running it twice doesn't double-install.

8. Extending the network — adding a new agent

The 5 newer agents (JARVIS, TANK, FLY GUY, BOB, STICKS) use a thin shim pattern. Follow it for new ones.

Step-by-step

1. **Add the system prompt** in `agents/prompts.js`:

```
`js newagent: You are NEWAGENT — your role.
```

```
${BUSINESS_CONTEXT}
```

```
OUTPUT: ... , `
```

1. **Create the launcher script** `newagent.js` at the install root:

```
``js #!/usr/bin/env node const { runAgent } = require('./lib/runner'); const prompts =  
require('./agents/prompts'); const { repl } = require('./lib/agent-cli');
```

```
repl({ name: 'NEWAGENT', system: prompts.newagent, greeting: 'Ask NEWAGENT anything.' }); ``
```

`lib/agent-cli.js::repl` handles the interactive loop; you just provide `name`, `system`, and `greeting`.

1. **Register it** in `agents/registry.json` under `agents`:

```
``json "newagent": { "script": "newagent.js", "role": "one-line description" } ``
```

1. **Add a smoke-test case** in `test-all-agents.js::LIVE_CASES`:

```
``js { id: 'newagent', prompt: 'MODE: SMOKE_TEST\nShort task. One line only.' }, ``
```

1. **Run the tests** — expect `live:newagent` to pass:

```
``powershell node test-all-agents.js --live ``
```

1. **Add a Desktop launcher** (manual) — copy any existing `.bat` in `Desktop\AIXMOS-LAUNCHERS\` and change the script name.
-

9. Editing agent prompts

All prompts live in `agents/prompts.js`. The `BUSINESS_CONTEXT` block at the top is interpolated into every agent's system prompt — change it once and every agent updates.

Voice rules that apply to customer-facing agents (CHUMMO, FLY GUY):

- Friend first, name first, one CTA.
- SMS under 160 chars when possible.
- Never corporate openers ("We are pleased to...", "Dear valued customer").
- Email = subject + body; SMS = text only.
- End every output with the agent's handoff marker (`MOOSE_HANDOFF:` , `CAPTAIN_HANDOFF:` , etc.).

After editing, run `node test-all-agents.js --live` to confirm the prompts still parse and produce sensible output.

10. Architecture

File layout

```
AIXMOS-AGENTS/
├─ orchestrator.js      CHUMMO + cross-agent menu
├─ moose.js             MOOSE standalone
├─ vision.js           VISION standalone
├─ briefing.js         Morning brief generator
├─ jarvis.js, tank.js, ... Thin shims using lib/agent-cli.js
├─ lib/
│  ├─ llm.js           ★ Dual-backend LLM abstraction (Claude / Ollama)
│  ├─ runner.js        Shared runAgent – delegates to lib/llm.js
│  ├─ env.js           Layered .env loader
│  ├─ agent-cli.js     Interactive REPL for shim agents
│  ├─ tank-docker.js   Docker availability check
│  └─ sticks-alerts.js Supabase overdue scan
├─ agents/
│  ├─ prompts.js       System prompts (10 agents + BUSINESS_CONTEXT)
│  └─ registry.json     Agent metadata + infrastructure paths
├─ config/
│  └─ aixmos.env.example Reference env file
├─ _installers/        Bundled Node.js MSI for fresh PCs
├─ node_modules/       Vendored deps (@anthropic-ai/sdk)
├─ aixmos-state.json   Per-agent runtime state (preserved across updates)
├─ package.json        npm scripts for every agent
├─ START-HERE.bat      ★ Top-level entry point (install / portable / offline)
├─ install-windows.bat Installer (mirrors tree to C:, creates launchers)
├─ RUN-FROM-USB.bat     Portable launcher (no install)
├─ OFFLINE-MODE-SETUP.bat Ollama install + model pull + env wiring
├─ RUN-AGENT-TESTS.bat Smoke test entry point
└─ test-all-agents.js Smoke test (structure + live API)
```

The LLM abstraction — `lib/llm.js`

Every agent that needs to think calls `generate({ system, prompt, maxTokens })` from `lib/llm.js`. That function reads `AIXMOS_LLM_BACKEND` and routes to either `callClaude` (uses `@anthropic-ai/sdk`) or `callOllama` (POSTs to `${OLLAMA_HOST}/api/chat`). `auto` mode tries Claude first and falls back to Ollama on any error.

Adding a new backend (e.g. Mistral.ai, vLLM, llama.cpp HTTP server) means adding one `callX` function + one `if (mode === 'x')` branch in `generate`. Everything downstream stays the same.

Per-agent flow

```
user clicks WONDER-WOMAN.bat
→ node wonderwoman.js
  → require('./lib/agent-cli').repl({name, system, greeting})
    → reads user input, builds prompt
    → require('./lib/runner').runAgent({system, userPrompt, maxTokens})
      → require('./lib/llm').generate({system, prompt, maxTokens})
        → callClaude (online) | callOllama (offline)
    → prints response, extracts handoffs, updates aixmos-state.json
  → loops
```

11. Reference — what the smoke test checks

`node test-all-agents.js --live` runs **49 checks**:

- 23 × `syntax:<file>` — every `.js` file must `node --check` clean.
- 2 × `require:<module>` — `state` and `prompts` must load.
- 1 × `state:roundtrip` — write → read state cycle.
- 1 × `registry:load` — `agents/registry.json` parses.
- 10 × `registry:<agent>:<script>` — every registered agent's script exists.
- 10 × `live:<agent>` (*when* `--live`) — every agent responds via the live LLM backend.
- 2 × `infra ( tank:docker-available , sticks:env )` — environment-only sanity checks.

Without `--live` it's 39 checks (no live LLM calls).

12. Where to get help

- **Agent prompts:** `agents/prompts.js` — every voice + output format lives here.
- **Infrastructure docs:** `INFRASTRUCTURE-QUICKSTART.md` and `README-AI-BRAIN.txt`.
- **Anthropic console:** <https://console.anthropic.com> (API keys, billing).
- **Ollama docs:** <https://ollama.com/library> (model catalogue, pull commands).

— END —